VB6 to AWS Migration

Project Work Guide

What the Client Gives You · What You Do · What You Deliver

Project Period: March – November 2026 | Target: Minimal & Risk-Mitigated Migration

Overall Project Flow

The project runs in two phases. First you analyze, then you migrate. Each phase has clear inputs (what you receive) and outputs (what you produce).

Receive Files & Access	Analyze & Document	Set Up AWS	Migrate Database	Move & Configure App	Test & Fix Issues	Deploy & Handover

What the Client Will Give You

On project kickoff, the client will hand over these items. For each item, you need to understand what it is and what your responsibility is.

#	Task / Deliverable	What You Do	Status
1	**VB6 Project Files (.vbp, .frm, .bas, .cls)**	Open in VB6 IDE, read the code, understand every screen and function, map the app logic	**Analyze**
2	**Compiled EXE file**	Run and observe the app behavior. Note every feature, screen, and action the app performs	**Analyze**
3	**INI / Config files**	Read every setting — DB server name, port, username, file paths. These must be updated for AWS	**Analyze**
4	**DLL and OCX files (dependencies)**	List all of them, check versions, confirm which ones are registered in Windows registry	**Analyze**
5	**Oracle 18c database access / dump**	Understand table structure, export data using Data Pump, prepare for import into Oracle 19c on AWS	**Analyze**
6	**VPN or Remote Desktop access (current server)**	Observe live environment. Note all paths, drives, printers, environment variables before changing anything	**Analyze**
7	**List of client PCs that run the app**	Count machines, check Windows versions, plan how to redistribute the updated EXE after migration	**Analyze**
8	**Any existing documentation (if**	Read everything — system diagrams, user manuals, past	**Analyze**

#	Task / Deliverable	What You Do	Status
	available)	issue logs. Gaps here mean more investigation for you	

Important: Even if source code is partially missing, you can still analyze the compiled EXE using tools like Dependency Walker and Resource Hacker. Document all gaps clearly in your Current State Analysis Report.

🔍 Phase 1 — Current State Analysis (March – May 2026)

This phase is all about understanding the existing system before touching anything. You are a detective here. No changes yet — only investigation and documentation.

What you do day-to-day in Phase 1:

#	Task / Deliverable	What You Do	Status
1.1	**VB6 Application Analysis**	Open all .vbp, .frm, .bas files. Map every screen, button, function and DB call. Create a screen-by-screen inventory	**Analyze**
1.2	**Database Structure Analysis**	Connect to Oracle 18c. List all tables, views, stored procedures. Understand data relationships	**Analyze**
1.3	**Dependency Audit**	Run Dependency Walker on the EXE. List every DLL/OCX. Check which are registered, which are missing	**Analyze**
1.4	**Connection String Documentation**	Find every connection string in code and INI files. Document server name, port, DB name, username	**Analyze**
1.5	**Environment Documentation**	Note all file paths, network drives, environment variables used by the app on current server	**Analyze**
1.6	**Risk Identification**	List everything that could break during migration — missing source, 32-bit DLLs, hardcoded paths, etc.	**Analyze**
1.7	**Current State Analysis Report**	Write and deliver the formal report summarizing all findings, risks, and recommendations	**Deliver**
1.8	**Migration Plan Document**	Based on findings, write a step-by-step plan for the actual migration in Phase 2	**Deliver**

The Current State Analysis Report is your most important deliverable in Phase 1. It is the foundation for everything in Phase 2. Be thorough — missed items here cause problems later.

☁ Phase 2 — Migration Execution (May – November 2026)

Now you actually do the migration. This splits into two options. You may be asked to estimate or execute either or both.

Option 1 — Minimal Migration (Lift & Shift)	Option 2 — Risk-Mitigated (With Minor Fixes)
Move the app to AWS with the least possible changes. Just get it running on EC2.	Same as Option 1, but also fix known issues in the code to reduce future risk.
Lower effort, faster delivery, higher risk of issues post-migration	More effort, longer timeline, but more stable and reliable result

Phase 2 Task Checklist (Both Options):

#	Task / Deliverable	What You Do	Status
2.1	**Set Up AWS EC2 Instance**	Launch Windows Server EC2. Configure VPC, Security Groups, RDP access, EBS storage. Test connectivity	**Deliver**
2.2	**Install Oracle 19c on EC2**	Install Oracle 19c on the EC2 instance. Configure listener, TNS names, ports	**Deliver**
2.3	**Migrate Oracle Database (18c → 19c)**	Export data from old Oracle 18c using Data Pump. Import into new Oracle 19c. Verify all tables and data	**Deliver**
2.4	**Register DLL/OCX Dependencies**	Copy all DLL/OCX files to EC2. Register them using regsvr32. Verify with Dependency Walker	**Deliver**
2.5	**Copy VB6 Application to EC2**	Copy EXE and all supporting files to EC2 server. Recreate the folder structure exactly	**Deliver**
2.6	**Update INI / Config Files**	Update DB server name, port, file paths in all config/INI files to match the new AWS environment	**Deliver**
2.7	**Option 2 Only: Edit VB6 Source Code**	Make minor code fixes — update connection strings, fix hardcoded paths, resolve known compatibility issues	**Deliver**
2.8	**Option 2 Only: Recompile EXE**	Recompile the fixed VB6 source into a new EXE file. Verify it compiles without errors	**Deliver**
2.9	**End-to-End Testing**	Launch the app on EC2. Test every screen and function. Confirm DB reads/writes work correctly	**Deliver**
2.10	**Bug Fixing & Issue Resolution**	Fix anything that breaks during testing — missing DLLs, wrong paths, DB connection errors	**Deliver**
2.11	**Distribute Updated EXE to Client PCs**	Package the new EXE + config files. Deploy to each client PC. Test on at least 2-3 machines	**Deliver**
2.12	**Performance & Stability Check**	Run the app for a period under normal use. Monitor for crashes, slow queries, connection drops	**Deliver**

📋 Full List of Deliverables — What You Give Back to the Client

These are the formal documents and files you must produce and hand over at the end of the project. Think of these as your final exam.

#	Deliverable	What It Contains	When
D1	**Current State Analysis Report**	Full analysis of the existing VB6 app, database, dependencies, file structure, risks, and gaps	**End of Phase 1**
D2	**Risk Register**	List of all identified risks, their likelihood, impact, and how you plan to handle each one	**End of Phase 1**
D3	**Migration Plan**	Step-by-step execution plan for Phase 2 — tasks, timeline, responsibilities, rollback plan	**End of Phase 1**
D4	**AWS Environment (EC2 + Oracle 19c)**	The actual live server on AWS with the database migrated and the app running	**During Phase 2**
D5	**Updated EXE + Config Files**	The migrated application files — ready to be deployed to client PCs	**During Phase 2**
D6	**Test Results Report**	Evidence that the app works correctly after migration — test cases, results, issues found and fixed	**End of Phase 2**
D7	**Deployment Guide**	Instructions for how to install and configure the app on a new client PC	**End of Phase 2**
D8	**Handover Document**	Summary of everything done, architecture of the new environment, maintenance notes, contact info	**End of Phase 2**

⚠ Common Issues You Will Face & How to Handle Them

Common Problem	Why It Happens	How to Fix It
VB6 source code is partially missing	Old projects often lose source files over time	Analyze the compiled EXE using Dependency Walker and Resource Hacker. Document gaps clearly
DLL/OCX fails to register on EC2	Windows Server version mismatch or wrong bitness (32 vs 64)	Use regsvr32 explicitly for each file. Check Event Viewer for errors. Use Dependency Walker to verify
Oracle connection fails after migration	TNS configuration, ODBC driver not installed, or wrong connection string	Check tnsnames.ora, listener.ora. Reinstall Oracle ODBC driver. Test with a simple SQL*Plus connection first
App works on EC2 but not on client PCs	Missing runtime files or wrong paths configured in INI files	Ensure MSVBVM60.DLL and all OCX files are distributed. Check all paths in config files
Hardcoded file paths break on new server	Old code has paths like C:\Server\App\data which do not exist on EC2	Find all hardcoded paths in VB6 source. Replace with configurable INI file settings
Data missing or corrupt after DB migration	Oracle Data Pump errors or charset mismatch between 18c and 19c	Always verify row counts before and after import. Run integrity checks. Keep a backup of 18c export

Keep this document with you throughout the project. Update the status column as you complete each task.